

SMART CONTRACT AUDIT REPORT

DATE : 26-05-2026

Executive Summary

- This audit was conducted by Mahin on 26-05-2026 for the DEHIX Web3 freelance and identity ecosystem. The assessment covered multiple smart contracts related to escrow management, staking systems, treasury fee handling, and Soulbound Token infrastructure. The review process included manual security analysis along with automated tooling such as Slither and Aderyn.

- During the review, several High and Medium severity vulnerabilities were identified. The most critical findings included Soulbound Token transfer bypasses, escrow voting replay risks, fee-on-transfer accounting desynchronization issues, and denial-of-service vectors caused by unbounded loops. These issues could potentially lead to unauthorized token transfers, locked funds, governance manipulation, and protocol instability if not properly mitigated.

- Additionally, multiple Low and Informational findings related to storage optimization, input validation, gas efficiency, and administrative centralization risks were identified. Remediation guidance and security recommendations were provided for all findings to strengthen the protocol's overall security posture, operational reliability, and maintainability.

Table of Contents

3 Escrow Contract

4 FreeLancer Contract

5 FreeLancerSoulBoundToken

6 DehixFeePool

7 InterviewerSoulBoundToken

8 NDASoulBoundToken

9 StakingRewards

Target Contract: EscrowContract.sol

Lines of Code (LoC): ~120 lines (excluding imports).
Language: Solidity Compiler version: ^0.8.28
External Dependencies: OpenZeppelin Contracts (IERC20, SafeERC20, ReentrancyGuard)

[M-01] Denial of Service (DoS) in Token Transfer Loop via Unbounded Array

- Severity : Medium

Description

In _transferTokens, the contract loops through the entire e.tokens array to transfer out accumulat balances:

```
for (uint256 i = 0; i < e.tokens.length; i++) {

    address token = e.tokens[i];

    uint256 amount = s_tokenBalances[_escrowId][token];
```

Because a client can call depositToken with an arbitrary number of unique ERC20 tokens , the e.toker array can grow indefinitely.

Impact
If the e.tokens array grows too large, the gas cost to iterate through the loop will exceed the block gas limit. This will permanently brick the releaseFunds, refundFunds, and resolveDispute function for that specific escrow ID, effectively locking up all deposited assets inside the contract. Furthermore, a malicious counterparty could deliberately spam small deposits of different tokens to freeze the escrow

Remediation
Avoid iterating through unbounded arrays to execute transfers. Instead of transferring all tokens automatically in a single transaction, implement a pull-based withdrawal pattern where users claim individual tokens, or enforce a strict configuration limit on the maximum number of unique tokens permitted per escrow contract (e.g., require(e.tokens.length <= 5, "TooManyTokens");).

[L-01] Storage Inefficiency and Plain-text Storage of Escrow IDs on-chain

- Severity : Low

Description

The contract uses a raw string data type for _escrowId across both storage mappings and function arguments



Storing and processing dynamically-sized strings on-chain is highly gas-inefficient compared to fixed-size types. Additionally, if the IDs represent sensitive internal database reference keys, exposing them as plain-text strings visible to everyone on-chain may leak private operational meta-data.

Impact

- High Gas Consumption: Dynamic strings require multiple storage slots and heavy computational overhead during runtime execution.
- Information Leakage: Plain text tracking exposes business identifiers to the public blockchain ledger unnecessarily.

Remediation
Switch from string to a fixed size bytes32 type for the escrow identifiers. Client applications can hash the human-readable string identifier off-chain using keccak256 before interacting with the smart contract:

[Informational] Missing Input Validation and Structural Dead Weight in Escrow Creation

- Severity : Informational

Description

When calling createEscrow, the system initializes an empty address[] tokens array explicitly inside the storage struct implicitly. However, the initial creation phase provides no capability to pre-register tokens or define authorized payment assets. The depositToken function handles pushing new entries autonomously later on anyway.

Impact

Clean codebase sanitation concern. The initialization flow contains non-functional design configurations that can lead to confusion for external integrators looking at the structural layout of the createEscrow function parameters

Remediation

Remove any confusion by adjusting documentation comments or validating inputs cleanly, confirming that the token array handles tracking dynamically via deposits, rather than expecting parameters at initialization.

- **Target Contract: FreelancerContract.sol**
- **Lines of Code (LoC): ~200 lines (excluding imports).**
- **Language: Solidity (Compiler version: ^0.8.28)**
- **External Dependencies: OpenZeppelin Contracts (IERC20, SafeERC20, ReentrancyGuard)**

[H-01] Critical Voting State Persistence Allows Instant Replay and Unauthorized Fund Draining

- Severity : high

Description

When an escrow is finalized via releaseFunds or refundFunds, the contract zeroes out the depositedAmount but fails to clear or deactivate the escrow ID, delete the oracle vote tallies, or clea the s_hasVoted checklist:

```
);
require(_majorityVote(_escrowId, true), "MajorityVoteFailed");
uint256 amountToRelease = escrow.depositedAmount;
require(amountToRelease > 0, "NoFunds");
escrow.depositedAmount = 0;
```

If a business creates a new escrow utilizing an identical _escrowId string (or if an identifier is reused globally across the protocol space), the historical votes stored inside s_releaseVotes, s_refundVotes, and s_hasVoted remain completely intact from the previous project cycle.

Impact

- Instant Dispute Hijacking: If the previous project reached a majority release vote decision , the new escrow immediately inherits that exact state profile.
- Capital Theft Vector: As soon as the business initializes and deposits funds into the reused escrow ID, a malicious actor or an authorized caller can instantly trigger releaseFunds or refundFunds on day one without waiting for any new oracle deliberation process.

Remediation

Implement a explicit structural deletion step inside termination entry points or track a clear lifecycle flag (e.g., enum EscrowStatus { None, Active, Finalized }). Prevent rewriting existing keys o explicitly clear accumulated vote structures upon close:

[M-01] Global Oracle Verification Loophole Disregards Specific Escrow Selection Context

- Severity : Medium

Description

When an escrow instance is created, a dedicated array of _votingOracles is defined specifically to manage that case. However, the validation within the vote() entry point checks only whether the sende is registered as a global system oracle:

```
function vote(
    string calldata _escrowId,
    bool _release
) external onlyOracle {
    require(!s_hasVoted[_escrowId][msg.sender], "AlreadyVoted");
    s_hasVoted[_escrowId][msg.sender] = true;
    if (_release) {
        s_releaseVotes[_escrowId]++;
    } else {
        s_refundVotes[_escrowId]++;
    }
    emit Voted(_escrowId, msg.sender, _release);
}
```

It fails to verify if msg.sender actually belongs to the assigned votingOracles collection saved inside the specific Escrow data structure.

Impact

Any approved protocol oracle in the system can interject and cast a vote on any active escrow dispute , irrespective of whether they were assigned to monitor that project. This dilutes the purpose of defining specific oracle groups during creation and disrupts consensus agreements

Remediation

Enforce a localized lookup check inside vote() to guarantee the calling address resides inside the targeted escrow's designated panel. To keep it gas efficient without iterating loops, consider mapping oracle authorization to individual escrow scopes during instantiation:

[M-02] Accounting De-synchronization via Fee-on-Transfer Tokens Locks Escrow Lifecycles

- Severity : Medium

Description

The depositFunds method calculates balances using internal tracking state addition relative to raw execution inputs:

```
escrow.depositedAmount = 0;
escrow.tokenAddress.safeTransfer(
    escrow.freelancerAddress,
    amountToRelease
);
emit FundsReleased(
```

If the designated settlement currency charges a fee directly during asset transport (e.g., deflationary tokens like PAXG or USDT under specific configurations), the absolute volume landed inside the destination address will be marginally less than the tracking parameter _amount

Impact

When releaseFunds or refundFunds gets called, the function reads escrow.depositedAmount and attempts to route that full volume back out. Because the physical balance in the contract is lower than the recorded value, the safeTransfer step will revert due to insufficient balance, indefinitely freezing the escrow's assets.

Remediation

Calculate total funds received dynamically by assessing balance differentials directly before and after execution:

[L-01] System Denial of Service (DoS) Risk via Unbounded Oracle Initialization Loop

- Severity : Low

Description

Inside createEscrow, the contract iterates through the provided _votingOracles input array to validate that every entry maps to a globally registered system oracle:

```
) external {
    require(bytes(_escrowId).length > 0, "EmptyEscrowId");
    require(_votingOracles.length % 2 != 0, "InvalidOracleCount");
    for (uint256 i = 0; i < _votingOracles.length; i++) {
        require(s_oracles[_votingOracles[i]], "NotAValidOracle");
    }
```

Impact

If an excessive number of oracles are appended to the configuration parameters, the transaction initialization gas cost will scale linearly (O(N) computational complexity). If it expands past acceptable thresholds, block resource restrictions could break construction reliability.

Remediation

Impose a strict maximum limit restriction check against incoming array sizes (e.g., require(_votingOracles.length <= 15, "ExceedsMaxOracles");) to guarantee safe execution windows

[L-02] Storage Inefficiency and Plain-text Storage of IDs on-chain

- Severity : Low

Description

Dynamic strings (string) are utilized to represent structural mapping index variables across keys (projectId, freelancerId, businessId, escrowId).



```
// --- STATE VARIABLES ---
struct Project {
    string projectId;
    bool isActive;
    mapping(address => bool) appliedFreelancers;
}
```

Impact

High gas expenditure during write/update procedures and unnecessary raw string exposure within global consensus storage layouts.

Remediation

Transition identity structures over to fixed-size bytes32 slots instead of dynamic text wrappers.

[M-01] Missing Duplicate Verification Triggers Continuous Unchecked Achievement Spanning

- Severity : Medium

Description

The internal execution layer _checkAndUnlockAchievements evaluates threshold boundaries dynamically every time updateReputation runs:

```
// Check for HIGHLY_RATED (average rating >= 90)
if (profile.averageRating >= 90) {
    _unlockAchievementInternal(
        _tokenId,
        Achievement.HIGHLY_RATED,
        "Achieved 90+ rating"
    );
}
```

The routine completely neglects to verify whether the target _tokenId has already unlocked the matching evaluation parameter

Impact

Every time an admin updates an eligible user's profile metadata (e.g., changing completion volume while maintaining a 95% evaluation rating), the system pushes an identical duplicate milestone into the tokenAchievements storage tracking array. This leads to catastrophic storage array stuffing, causing massive gas spikes when calling data-read targets like getAchievements().

Remediation

Introduce a quick bitmask or tracking layout to check off whether specific categorical indexes have already been claimed by the token profile before invoking the internal append mechanics:

[L-01] Unbounded Loop Processing in Verification Iteration

Severity : low

Description

The contract uses an unbounded manual for loop inside _checkAndUnlockAchievements to calculat

total valid competencies dynamically off the string-backed skillVerified variable collection



```
for (uint256 i = 0; i < profile.skillVerified.length; i++) {
    if (profile.skillVerified[i]) {
        verifiedSkillCount++;
    }
```

Impact
If a freelancer receives a high volume of skill endorsements over time, processing reputation assessments through updateReputation will scale linearly in gas costs (O(N)), introducing potential block gas limit denial of service conditions

Remediation
Replace loops with hot state tracking variables. Increment a separate verifiedSkillsCounter tracking uint within endorseSkill and decrement it within revokeSkillEndorsement so that calculations are always instantly readable at a constant $O(1)$ runtime speed.

- **Target Contract: DEHIXFeePool.sol**
- **Lines of Code (LoC): ~60 lines.**
- **Language: Solidity (Compiler version: ^0.8.24)**
- **External Dependencies: OpenZeppelin Contracts (Ownable)**

[L-01] Centralization Risk: Missing Timelock Protection on Critical Parameter Controls

- Severity : low

Description

Administrative configuration methods such as setMintFee and setSBTContract execute parameter changes instantly the moment they are called by the onlyOwner role:

```
function setMintFee(uint256 newFee) external onlyOwner {
    uint256 oldFee = mintFeePerCertificate;
    mintFeePerCertificate = newFee;
    emit MintFeeUpdated(oldFee, newFee);
}
```

Impact

- Lack of User Defense: If the admin private key is ever compromised, a malicious actor could instantly change mintFeePerCertificate to an astronomical price. Unsuspecting users interacting with the frontend platform or relying on automation layers could have their deposited balance drained instantly via malicious deduction calls.

- Malicious Rugpull Exposure: Without a delay window, an aggressive or corrupt admin can route the target contract authorization over to a malicious proxy address instantly, tricking client integrations and stealing platform assets.

Remediation

Integrate a programmatic Timelock delay (e.g., 48 hours) for critical state adjustments. This can be accomplished by routing governance control over these functions through an external Timelock contract, giving users a clear notice period to safely withdraw their funds if an unexpected or malicious setting update is requested.

[M-XX] Missing State Synchronization: Direct Native Token Deposits Bypass totalFeesCollected Accounting

- Severity : Medium

Description

The contract maintains a global state variable totalFeesCollected to track the gross volume of fe revenue processed by the protocol over its operating lifecycle:

```
receive() external payable {
    userBalance[msg.sender] += msg.value;
    emit FeeDeposited(msg.sender, msg.value);
}
```

Impact

- Broken Financial Metrics: The totalFeesCollected state variable will lag behind real-time values failing to accurately reflect the total protocol volume.
- Downstream Integration Failures: If your company integrates administrative tracking dashboards, analytical frontends, or automated reward distribution/treasury contracts that depend on totalFeesCollected to measure protocol revenue, their execution logic will calculat incorrect distributions due to stale on-chain metric data.

Remediation

Ensure that any entry point altering or accumulating system deposits updates the protocol's global tracker synchronously

- **Target Contract: InterviewSoulboundToken.sol**
- **Lines of Code (LoC): ~85 lines.**
- **Language: Solidity (Compiler version: ^0.8.30)**
- **External Dependencies: OpenZeppelin Contracts (ERC721)**

[H-01] Identity Spoofing: Soulbound Mechanism Bypass via Unprotected 3-Argument safeTransferFrom Interface

- Severity : high

Description

The contract explicitly overrides the standard ERC-721 transfer and approval functions to prevent tokens from changing hands:

```
function transferFrom(address, address, uint256) public pure override {
    revert("Soulbound: transfer disabled");
}

function safeTransferFrom(address, address, uint256, bytes memory) public pure override {
    revert("Soulbound: transfer disabled");
}
```

However, OpenZeppelin's base ERC721 implementation natively exposes two independent public overrides for safe transfers:

1.safeTransferFrom(address from, address to, uint256 tokenId, bytes memory data)
2.safeTransferFrom(address from, address to, uint256 tokenId)

Because the contract completely omits overriding the 3-argument signature variation, a user can bypass the revert statement entirely by calling safeTransferFrom(from, to, tokenId). The transactio passes through the contract into OpenZeppelin's native baseline logic, completing the transfer successfully.

Impact

- Total Breakdown of Soulbound Logic: The token is completely transferable, violating the core requirement of a Soulbound Token ecosystem.
- Reputation & Credential Trading: High-performing participants can transfer or sell their interview completion records to third-party accounts on secondary marketplaces, completely destroying the integrity of the platform's validation records.

Remediation

Instead of attempting to block public function entry points individually, intercept all state alterations at the core bottleneck by overriding the internal transfer hook. Depending on the specific version of OpenZeppelin being utilized, override either _update (for OpenZeppelin v5.x) or _beforeTokenTransfer (for OpenZeppelin v4.x) to completely disable transfers except for minting and burning:

- **Target Contract: InterviewSoulboundToken.sol**
- **Lines of Code (LoC): ~85 lines.**
- **Language: Solidity (Compiler version: ^0.8.30)**
- **External Dependencies: OpenZeppelin Contracts (ERC721)**

[H-01] Identity Spoofing: Soulbound Mechanism Bypass via Unprotected 3-Argument safeTransferFrom Interface

- Severity : high

Description

The contract attempts to enforce soulbound behavior by overriding two specific ERC721 transfer signatures:

```
function transferFrom(address, address, uint256) public pure override {
    revert("SoulBound: Tokens cannot be transferred");
}

function safeTransferFrom(
    address,
    address,
    uint256,
    bytes memory
) public pure override {
    revert("SoulBound: Tokens cannot be transferred");
}
```

Because the standard 3-argument version safeTransferFrom(address from, address to, uint256 tokenId) is omitted from the overrides, the base OpenZeppelin implementation remains fully operational and accessible.

Impact

Users can freely bypass the soulbound constraints by calling the 3-argument signature. This allows legal NDAs to be transferred or traded on secondary markets, completely breaking the privacy and validation architecture of the platform.

Remediation

Override the internal storage management hook to catch all token movements globally (both 3-argument and 4-argument public transfer paths).

[M-02] Invariant Breakdown: Missing Verification Validation Allows Creation of Empty or Malforme NDAs

- Severity : medium

Description

When a business owner initializes an agreement via createNDA, the execution flow accepts string inputs without validating their contents:



```
function createNDA(
    string calldata _content,
    address _freelancer,
    uint256 _durationDays
) external onlyRole(BUSINESS_OWNER_ROLE) nonReentrant returns (uint256) {
    require(bytes(_content).length > 0, "EmptyContent");
    require(_freelancer != address(0), "InvalidFreelancer");
    require(_durationDays > 0, "InvalidDuration");

    uint256 tokenId = s_tokenIdCounter++;
    uint256 expirationTime = block.timestamp + (_durationDays * 1 days);

    ndas[tokenId] = NDA({
        id: tokenId,
        content: _content,
        businessOwner: msg.sender,
        freelancer: _freelancer,
        businessSignature: "",
        freelancerSignature: "",
        createdAt: block.timestamp,
        signedAt: 0,
        expirationTime: expirationTime,
        completionTime: 0,
        status: NDAStatus.Draft,
        burned: false
    });
```

Impact

A compromised or malfunctioning business account can spam the blockchain by minting empty string records or assigning legal responsibilities to the zero address (address(0)). Because tracking dynamic structures like businessOwnerNDAs and freelancerNDAs append these keys automatically, the system's global indexing logic will fill up with dead tracking references.

Remediation

Add strict input parameter validation statements to the entry execution parameters

[M-02] Precision Loss in rewardRate Calculation Breaks Low-Decimal Reward Distributions

- Severity : medium

Description

In notifyRewardAmount, the contract computes the distribution pacing speed using basic division



```
function withdraw(
    uint256 _amount
) external nonReentrant updateReward(msg.sender) {
    require(_amount > 0, "amount = 0");
    require(balanceOf[msg.sender] >= _amount, "insufficient balance");
    // Effects
    balanceOf[msg.sender] -= _amount;
    totalSupply -= _amount;
    // Interaction
    stakingToken.safeTransfer(msg.sender, _amount);
    emit Withdrawn(msg.sender, _amount);
}
```

Because Solidity does not natively support fixed-point decimals or floating-point numbers, integer division can easily suffer from rounding down to zero.
If the reward asset uses a lower decimal standard (such as USDC or USDT which use 6 decimals instea of 18) and the admin chooses a standard long-duration campaign (e.g., 30 or 90 days, measured in seconds), the value of _amount can easily be lower than the duration value

Impact

The math will round down to 0. This triggers the subsequent constraint.The transaction will revert instantly. This completely blocks the owner from setting up low-decimal token rewards or low-value incentive programs across long distribution windows

Remediation

To avoid rounding down to zero, multiply and scale up your reward rates using an internal precision multiplier (e.g., 1e18), and scale down only when calculating final individual payouts